# MongoDB & Mongoose

Beginner-Friendly Study Material • Concepts, Setup, CRUD, Examples & Viva Questions

**MongoDB**
A NoSQL document database that stores data in BSON-like documents.

**Mongoose**
A Node.js ODM (Object Data Modeling) library used to define schemas and work with MongoDB more conveniently.

Designed for students learning Node.js backend development and full-stack projects.

# 1. Learning Objectives

• Understand what MongoDB is and how it stores data.

• Explain collections, documents, fields, and the _id field.

• Understand Mongoose, schemas, models, validation, and connections.

• Build basic Create, Read, Update, and Delete (CRUD) operations.

• Connect a Node.js/Express application to MongoDB safely.

# 2. What is MongoDB?

MongoDB is a NoSQL, document-oriented database. Instead of storing data in rows and columns like a traditional relational database, it stores records as documents made of field-value pairs. Documents are grouped into collections.

## Important terms

• **Database:** Container for collections.

• **Collection:** Group of related documents; similar in purpose to a table.

• **Document:** One record, represented using BSON (a binary-encoded, JSON-like format).

• **Field:** A named value inside a document.

• **_id:** Unique identifier for each document; MongoDB creates one automatically if it is not supplied.

## Example document

```
{
  "_id": "generated ObjectId",
  "name": "Ananya",
  "age": 20,
  "course": "Computer Science",
  "active": true
}
```

## MongoDB vs relational databases

| Relational database | MongoDB |
|---|---|
| Table | Collection |
| Row / record | Document |
| Column | Field |
| Usually a fixed table structure | Flexible document structure |

# 3. What is Mongoose?

Mongoose is an ODM (Object Data Modeling) library for Node.js. It helps an application interact with MongoDB using JavaScript objects and provides schemas, models, validation, middleware, and convenient query methods.

## Why use Mongoose?

• Defines the expected structure of documents through a schema.

• Validates data before saving it.

• Provides model methods such as find(), findById(), create(), and findByIdAndUpdate().

• Supports defaults, timestamps, relationships/references, and middleware.

• Makes database code easier to organize in Node.js projects.

**Remember:** MongoDB is the database; Mongoose is a library used by your Node.js code to work with that database.

# 4. Setup

## Requirements

• Install Node.js and npm.

• Have a MongoDB server available locally, or create a MongoDB Atlas cluster.

• Create a project folder and open its terminal.

## Commands

```
mkdir mongoose-demo
cd mongoose-demo
npm init -y
npm install express mongoose dotenv
```

For a simple project, create these files: **server.js**, **.env**, and optionally a **models/Student.js** file.

## Environment variable

```
# .env
MONGODB_URI=mongodb://127.0.0.1:27017/studentdb
PORT=5000
```

For MongoDB Atlas, use the connection string supplied by Atlas instead of the local URI. Keep passwords and connection strings private; do not commit the .env file to a public repository.

# 5. Create a Mongoose Schema and Model

A schema describes the shape, data types, and validation rules for documents. A model is created from a schema and is used to perform database operations.

```
const mongoose = require("mongoose");

const studentSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true
    },
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true
    },
    age: {
      type: Number,
      min: 16,
      max: 100
    },
    course: {
      type: String,
      required: true
    }
  },
  { timestamps: true }
);

module.exports = mongoose.model("Student", studentSchema);
```

The model name is **Student**. Mongoose normally maps it to a plural, lowercase collection name such as **students**. The timestamps option adds createdAt and updatedAt fields.

# 6. Connect Express to MongoDB

```
require("dotenv").config();
const express = require("express");
const mongoose = require("mongoose");

const app = express();
app.use(express.json());

mongoose.connect(process.env.MONGODB_URI)
  .then(() => {
    console.log("MongoDB connected");
    app.listen(process.env.PORT || 5000, () => {
      console.log("Server is running");
    });
  })
  .catch((error) => {
    console.error("MongoDB connection failed:", error.message);
  });
```

This example starts the HTTP server only after the database connection succeeds. In a larger application, separate the database connection, routes, models, and server setup into different files.

# 7. CRUD Operations

CRUD means Create, Read, Update, and Delete. The following examples use the Student model defined above.

## Create — insert a student

```
const student = await Student.create({
  name: "Ananya Rao",
  email: "ananya@example.com",
  age: 20,
  course: "Computer Science"
});
```

## Read — get documents

```
const allStudents = await Student.find();
const oneStudent = await Student.findById(studentId);
const csStudents = await Student.find({ course: "Computer Science" });
```

## Update — change a document

```
const updatedStudent = await Student.findByIdAndUpdate(
  studentId,
  { age: 21 },
  { new: true, runValidators: true }
);
```

The new: true option returns the updated document. runValidators: true asks Mongoose to apply schema validators to the update.

## Delete — remove a document

```
const deletedStudent = await Student.findByIdAndDelete(studentId);
```

In real APIs, check whether the returned value is null. A null result usually means the requested ID did not match a document.

# 8. Complete Small Express API Example

This example assumes the Student model is saved in models/Student.js. It demonstrates common routes; add production-level authentication, authorization, logging, and stronger error handling before real deployment.

```
require("dotenv").config();
const express = require("express");
const mongoose = require("mongoose");
const Student = require("./models/Student");
```

```
const app = express();
app.use(express.json());

app.get("/api/students", async (req, res) => {
  try {
    const students = await Student.find();
    res.json(students);
  } catch (err) {
    res.status(500).json({ message: "Could not fetch students" });
  }
});

app.post("/api/students", async (req, res) => {
  try {
    const student = await Student.create(req.body);
    res.status(201).json(student);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

app.get("/api/students/:id", async (req, res) => {
  try {
    const student = await Student.findById(req.params.id);
    if (!student) return res.status(404).json({ message: "Not found" });
    res.json(student);
  } catch (err) {
    res.status(400).json({ message: "Invalid student ID" });
  }
});

app.put("/api/students/:id", async (req, res) => {
  try {
    const student = await Student.findByIdAndUpdate(
      req.params.id, req.body,
      { new: true, runValidators: true }
    );
    if (!student) return res.status(404).json({ message: "Not found" });
    res.json(student);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

app.delete("/api/students/:id", async (req, res) => {
  try {
    const student = await Student.findByIdAndDelete(req.params.id);
    if (!student) return res.status(404).json({ message: "Not found" });
    res.json({ message: "Student deleted" });
  } catch (err) {
    res.status(400).json({ message: "Invalid student ID" });
  }
});

mongoose.connect(process.env.MONGODB_URI)
  .then(() => app.listen(process.env.PORT || 5000,
    () => console.log("Server started")))
  .catch(err => console.error(err.message));
```

Note: In production, validate request bodies and allow only approved fields to be changed. Do not blindly trust all client-supplied data.

# 9. Common Mongoose Methods

| Method | Purpose |
|---|---|
| Model.create(data) | Create and save a document |
| Model.find(filter) | Find all matching documents |
| Model.findOne(filter) | Find the first matching document |

| | |
|---|---|
| Model.findById(id) | Find a document by its _id |
| Model.updateOne(filter, update) | Update one matching document |
| Model.findByIdAndUpdate(id, update, options) | Update by ID and optionally return updated document |
| Model.deleteOne(filter) | Delete one matching document |
| Model.findByIdAndDelete(id) | Delete by ID and return deleted document |

# 10. Validation, Defaults, and Timestamps

```
const productSchema = new mongoose.Schema({
  title: { type: String, required: true, trim: true },
  price: { type: Number, required: true, min: 0 },
  inStock: { type: Boolean, default: true }
}, { timestamps: true });
```

• **required:** field must be supplied.

• **min / max:** numeric bounds.

• **enum:** restrict a value to a list of allowed choices.

• **default:** value used when one is not supplied.

• **trim:** removes whitespace from the beginning and end of strings.

• **timestamps:** automatically manages createdAt and updatedAt.

Important: unique: true creates or requests a unique index; it is not a normal validator. Handle duplicate-key database errors in your application.

# 11. Async/Await and Error Handling

Database operations take time, so Mongoose methods are commonly used with async/await. Put them inside try/catch blocks so the API can return a suitable response when an operation fails.

```
app.get("/api/example", async (req, res) => {
  try {
    const data = await Student.find();
    res.json(data);
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: "Internal server error" });
  }
});
```

# 12. Common Errors and Fixes

• **MongoServerSelectionError / connection refused:** make sure MongoDB is running and the URI, host, and port are correct.

• **Authentication failed:** verify Atlas database username/password and database access permissions.

• **IP not allowed in Atlas:** check the project's Network Access rules; avoid opening access broadly for production.

• **ValidationError:** check required fields, types, and min/max constraints.

• **Duplicate key error:** check fields with unique indexes, such as email.

• **CastError / invalid ObjectId:** validate the ID format and handle invalid IDs.

• **Request body is undefined:** add app.use(express.json()) before the routes and send JSON with the correct Content-Type.

# 13. Best Practices

• Store connection strings and credentials in environment variables.

• Never commit .env files or expose database passwords.

• Validate user input and whitelist fields accepted by update routes.

• Use appropriate HTTP status codes: 200, 201, 400, 404, and 500.

• Use indexes for fields frequently searched; measure performance before adding many indexes.

• Use pagination for large result sets.

• Do not return sensitive fields in API responses.

• Back up important data and restrict database network access.

# 14. Quick Revision

| Question | Short answer |
|---|---|
| What is MongoDB? | A NoSQL document database. |
| What is Mongoose? | An ODM library for Node.js and MongoDB. |
| What is a schema? | A definition of document fields, types, and rules. |
| What is a model? | An object created from a schema to query and modify a collection. |
| What is CRUD? | Create, Read, Update, Delete. |
| What does find() do? | Returns documents matching a filter. |
| What does _id mean? | The unique identifier of a document. |
| Why use async/await? | To wait for asynchronous database operations in readable code. |

# 15. Viva / Practice Questions

• Define MongoDB and list two advantages.

• Differentiate a collection and a document.

• What is Mongoose and why is it used?

• Explain the difference between a schema and a model.

• What is the purpose of mongoose.connect()?

• Explain Create, Read, Update, and Delete with one method each.

• What is the purpose of required and default in a schema?

• Why should errors be handled with try/catch?

• What does findByIdAndUpdate() return when new: true is used?

• Why should a database connection string be kept in an environment variable?

# 16. Mini Practice Task

Create a Book model with title, author, price, and available fields. Then implement these routes:

• POST /api/books — add a book.

• GET /api/books — list all books.

• GET /api/books/:id — get one book.

• PUT /api/books/:id — update a book.

• DELETE /api/books/:id — delete a book.

Test each route with Postman, Thunder Client, or another API client. Confirm success responses and error cases.

# References for Further Learning

• MongoDB Manual: https://www.mongodb.com/docs/manual/

• Mongoose documentation: https://mongoosejs.com/docs/

• Express documentation: https://expressjs.com/

Prepared as introductory study notes. APIs and package behavior can change between versions; consult the official documentation for version-specific details.